

Rapport final du projet 4IM01- Filtres de "vesselness" et détection de structures tubulaires

Alexis Cordelle
Mohamed Boudouh

Septembre Octobre Novembre 2025

Problématique , Architecture globale du projet

L'article de base présente des filtres (par exemple de Frangi et de Sato) qui peuvent être utilisés pour segmenter les vaisseaux. Ils peuvent être interprétés comme des filtres qui sélectionnent des régions dans lesquelles les valeurs propres de la Hessienne vérifient des conditions particulières.

L'article explique que ces filtres ont deux inconvénients : Sensibilité au changement de contraste (causé par une réponse presque croissante en la norme des valeurs propres pour le filtre de Frangi par exemple^o , Manque d'uniformité de la réponse sur une section du vaisseau (qui peut être causé par la décroissance de la norme des valeurs propres en s'approchant des bords du vaisseaux)

On se propose dans ce projet d'implémenter numériquement le filtre de Frangi(2D et 3D) et celui décrit dans l'article(2D) en essayant d'apporter quelques adaptations à ce contexte. Une implémentation sur GPU a été également effectuée et a permis une réduction notable du temps d'exécution. Enfin, des métriques quantitatives ont été implémentées et ont servi comme outil de comparaison des deux filtres.

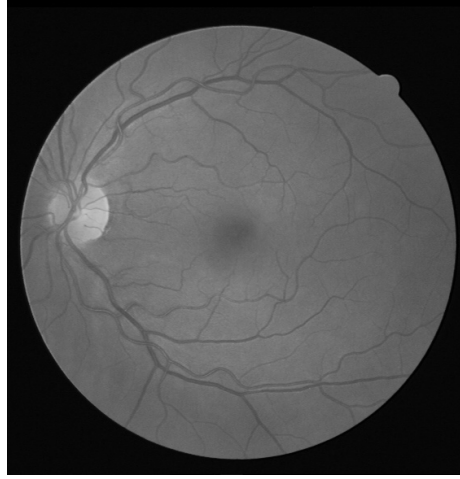


Figure 1: Image de vaisseaux oculaires

Premières améliorations de l'implémentation : Trouver un noyau de convolution plus adapté à la segmentation

La remarque sur laquelle est basée cette partie est que la fonction gaussienne qui sert dans l'article de noyau de convolution moyenne uniquement en fonction de la distance. On veut la remplacer par une fonction qui moyenne en fonction de la distance et de la direction afin d'avoir une image lisse (pour laquelle on peut calculer une matrice hessienne) mais aussi qui conserve le contraste entre les vaisseaux et les autres structures. Cette image montre par exemple que les vaisseaux (principaux) avant bifurcations ont une direction plus proche de l'horizontale que la verticale.

On peut par exemple utiliser une gaussienne dont la matrice de covariance **n'est pas scalaire**. En particulier, on veut qu'elle pénalise une direction plus qu'une autre.

Ci-dessous le résultat de l'utilisation d'une fonction gaussienne de matrice de covariance diagonale mais qui a une plus grande variance selon la direction x que selon y : $\sigma_y = 0.7\sigma_x$

Approches locales

Une autre approche a été essayée dans laquelle la moyenne autour de chaque pixel était faite dans une direction qui dépend de son information locale :

Algorithme 1 - On commence par l'algorithme le plus simple, juste prendre le max des dérivés selon plusieurs directions



Figure 2: Resultat d'un filtre de frangi basé sur une fonction gaussienne de variance plus grande selon l'axe horizontale que verticale ($\sigma_y = 0.7\sigma_x$)

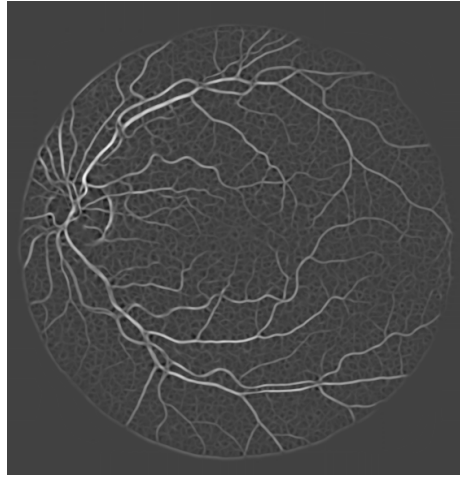


Figure 3: Résultat de l'algorithme 1

Algorithm 1 Direction de variation maximale

Require: $(g_{i,j})$ Image en niveaux de gris , $k > 0$

for $i, j \leq N$ **do**

for $d \leq k$ **do** calculer $(I \otimes G)((g_{i,j})$ dans une base résultante de la rotation de la base initiale d'un angle $\frac{2\pi d}{k}$

end for

 Conserver le résultat maximal pour le pixel concerné.

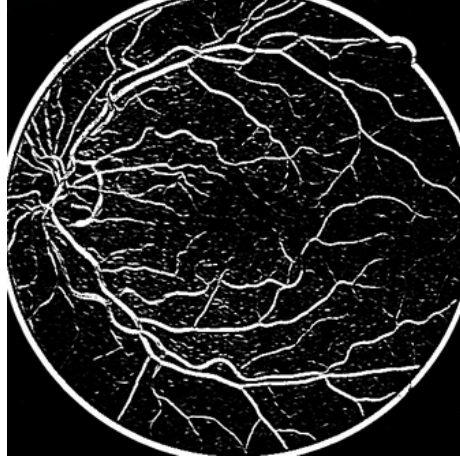


Figure 4: Résultat de l'algorithme 2

Algorithme 2

Algorithm 2 Pondération en fonction des espaces propres de la matrice Hessienne

Require: $(g_{i,j})$ Image en niveaux de gris ,

for $i, j \leq N$ **do**

for $t, s \leq N$ **do** Calculer $\|P_{i,j} - P(t, s)\|$ avec P matrice des vecteurs propres de la matrice hessienne dans la base canonique

end for

calculer $(I \otimes G \otimes W)((g_{i,j})$ Avec W une fonction de pondération qui dépend des normes calculés précédemment

Passage de l'image en couleur à l'image en gris

On modélise le passage le passage d'une image en couleurs RGB à une image en gris par une application de $\mathbf{R}^3$ vers $\mathbf{R}$.

.On peut chercher une fonction adaptée au problème de ségmentation dans le sens qu'elle "sépare" le mieux les structures qu'on veut sélectionner des autres structures présentes dans l'image. En particulier, si on ne garde que l'intensité du pixel ,on peut perdre l'information sur la couleur particulière des vaisseaux qui peut nous aider dans la ségmentation.

Certains critères peuvent etre définis pour nous aider dans le choix de cette fonction. Par exemple:

$$f = \underset{f: \mathbf{R}^3 \rightarrow \mathbf{R}}{\operatorname{Argmax}} (\sum_{x \in V} \sum_{y \notin V} (f(x))^2 - (f(y))^2) \quad (1)$$

V désigne l'ensemble des pixels contenus dans une partie de l'image représentant le vaisseau.

Cas d'une forme linéaire

Dans le cas des applications linéaires , le maximum dans (1) sur la sphère unité (pour la norme d'opérateur) est atteint en

$$f : x \rightarrow s^T x \text{ avec :}$$

$$s = 1 / (\| \sum_{x \in V} \sum_{y \notin V} (x - y) \|_2) \sum_{x \in V} \sum_{y \notin V} (x - y) \quad (2)$$

Remarque

Ce résultat décrit un cas particulier de l'analyse en composante principale(ACP)

En pratique , on ne connait pas V à cette étape ,on peut effectuer une ségmentation préliminaire pour avoir un ensemble "proche" de V pour ensuite choisir une forme linéaire plus fine.

Méthode CLAHE

Une méthode existante et implémentée ensuite est la méthode CLAHE. Le prétraitement avec clahé fonctionne relativement et permet de mettre en valeur certaines zones, au détriment d'autre. Plutôt une amélioration même si le bruit reste important, il a une texture différentes.



Figure 5: Résultat de la transformation linéaire de l'image en couleurs

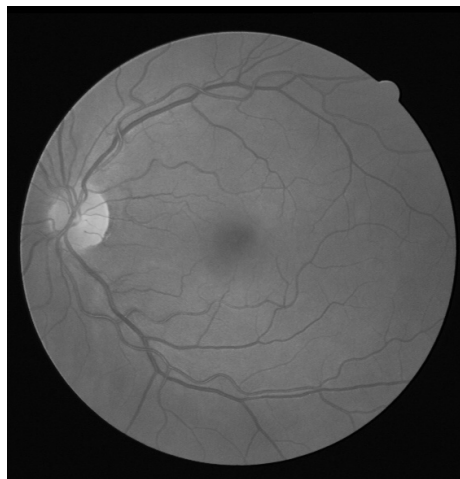


Figure 6: Résultat du passage de l'image en couleurs à l'image en gris par moyennage des valeurs des 3 canaux

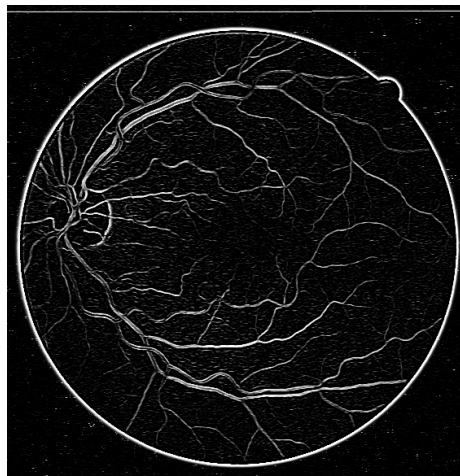


Figure 7: Résultat du filtre de frangi appliquée à l'image grise obtenue par moyenne des 3 canaux

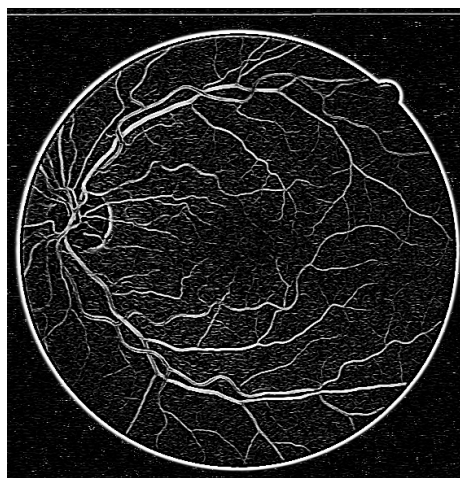


Figure 8: Résultat du filtre de frangi appliquée à l'image grise obtenue par la transformation linéaire(où on a voulu implémenter (2)) de la meme image en couleurs initiale

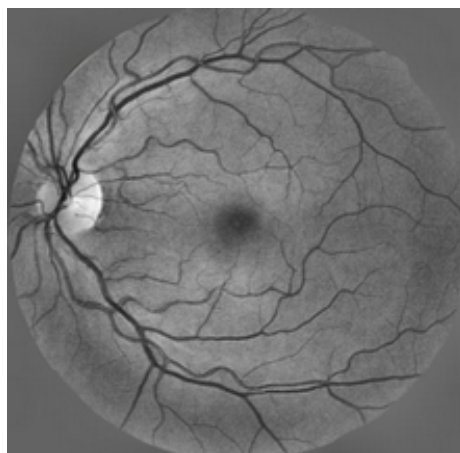


Figure 9: Résultat du prétraitement par la méthode Clahe

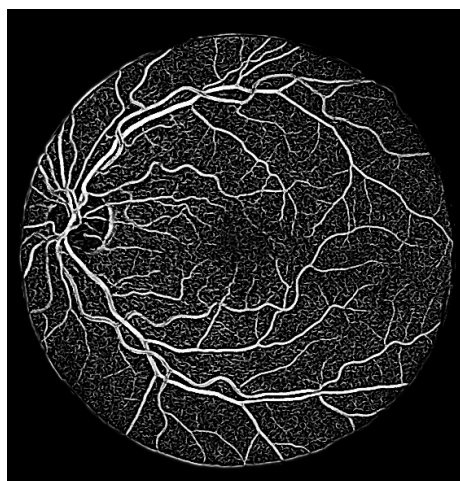


Figure 10: Résultat de l'application du filtre de Frangi après prétraitement par la méthode Clahe

Métriques d'évaluation

Contexte

Le papier de recherche de base a d'abord présenté des filtres antérieurement utilisé pour la segmentation de vaisseaux (Frangi et Sato par exemple), mais attire l'attention a 2 points faibles :

Manque d'uniformité sur une section du vaisseau (décroissance en s'éloignant du centre .

Sensibilité au changement du **contraste**

L'une des motivations de cette partie est de vérifier si le filtre proposé a effectivement une meilleure performance sur ces 2 points. Plus généralement , on pense que des métriques quantitatives seront plus facilement interprétables pour évaluer les résultats(en particulier la robustesse au bruit par exemple) ou pour choisir les paramètres du filtres (τ pour le filtre proposé par exemple)

Métriques évoqués dans l'article publié

L'article a évoqué trois métriques AUPRCC (Area under the precision recall curve) , SNR , median of normalized response inside the reference segmentation MEDNR.

AUPCR

Area under the precision recall curve est une métrique mentionné dans l'article. C'est une métrique d'évaluation de résultats de filtres de classification (en général binaire) qui se base principalement sur deux grandeurs :

Precision(treshhold) : proportion des vrais positifs par rapport aux positifs détectés si le seuil choisi est treshhold.

Recall(treshhold): proportion des positifs détectés par rapport aux vrais positifs.

Définition La grandeur estimée est la suivante : $\theta = - \int_{-\infty}^{\infty} Precision(treshhold) dRecall(treshhold)$

Elle pourrait ainsi etre interprétée comme une précision moyenne par rapport aux seuils possibles (pour la mesure particulière $1 - Recall(treshhold)$)

Remarque: Une implémentation de cette métrique suppose en général l'existence d'une ségmentation parfaite. Pratiquement,on a utilisé des ségmentations manuelles comme référence.

Remarque: Un article expliquant la métrique ausssi bien que quelques estimateurs empiriques est disponible sur le lien: https://pages.cs.wisc.edu/boyd/aucpr_final.pdf

MEDNR

MEDNR signifie "the mediane response of the normalized filter inside the reference segmentation. On pourrait interpréter cette métrique comme un indice

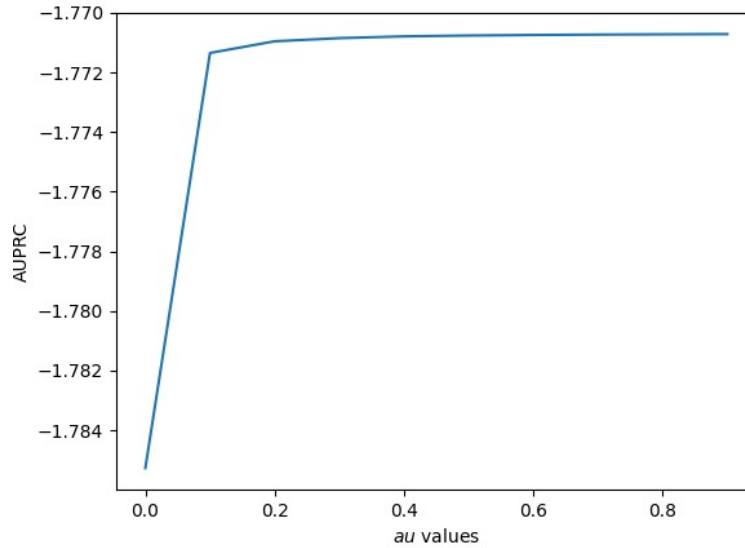


Figure 11: $\log(AUPRC) = f(\tau)$

qui indique si la segmentation se fait d'une manière certaine ou si elle est sensible au seuil choisi en aval du filtre.

SNR

Le bruit dans l'image segmentée n'étant pas facile à estimer dans ce contexte de segmentation par une approche spectrale. Nous décidons de l'estimer par la formule suivante:

$$SNR = \left(\frac{NumberofTruePositifs}{NumberoffalsePositifs} \right)^2$$

Application 1 : Choix des paramètres

Remarque Il est important de mentionner qu'une image de vaisseaux oculaires a été utilisée dans cette partie et que nous ne possédons pas de garantie que

Choix de τ pour le filtre proposé dans l'article

La figure suivante montre le résultat de $\log(AUPRC) = f(\tau)$ pour quelques valeurs de τ . L'utilisation de $\log$ est pour permettre une meilleure séparation des valeurs. Cette figure permet par exemple de justifier le choix $\tau = 1$

Il est important de souligner que ces résultats sont surprenant. Je pense qu'en particulier AUPRC ne s'intéresse pas aux faux négatifs, il peut donc valoriser une image complètement noire. Cette métrique reste donc insuffisante.

Choix des deux paramètres pour le filtre defrangi En se basant sur la

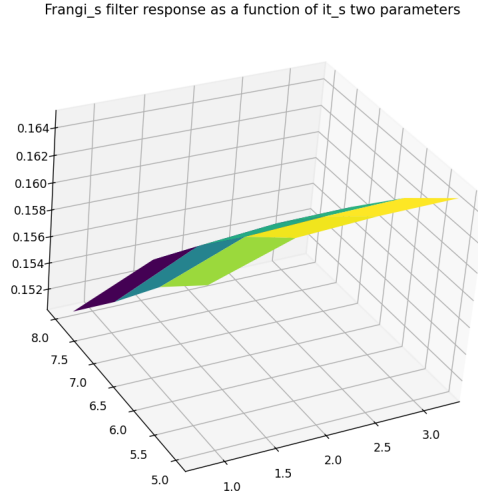


Figure 12: Résultat de AUPRC filtre de frangi en fonction de ses 2 paramètres

figure précédente , Les paramètres b et c peuvent être pris respectivement 0.6 et 7 . Une animation pourrait montrer l'évolution de la qualité de la segmentation plus clairement.

Application 2 : Test de robustesse face au changement de contraste

Nous choisissons de nous baser sur MEDNR vu qu'il décrit "la certitude avec laquelle l'algorithme arrive à classer les pixels" . Ce dernier aspect étant le plus sensible aux changements de contraste.

La figure 13 montre des particularités des 2 filtres. Notons que la partie décroissante de la courbe correspondante au filtre proposé dans l'article peut s'expliquer par le fait que le filtre peut prendre des valeurs (continuellement) petites même en diminuant la norme d'une valeur propre , ce qui me paraît impossible pour le filtre de Frangi. En particulier , le vaisseau peut paraître plus large pour le filtre proposé.

Application 3 : Test d'uniformité sur une section du vaisseau

Nous nous baserons sur la figure 13 . Effectivement, le fait d'avoir un MEDNR peut s'expliquer par la décroissance progressive de la réponse du filtre en s'approchant des bords. Il semble par contre que le filtre de Frangi a une décroissance plus

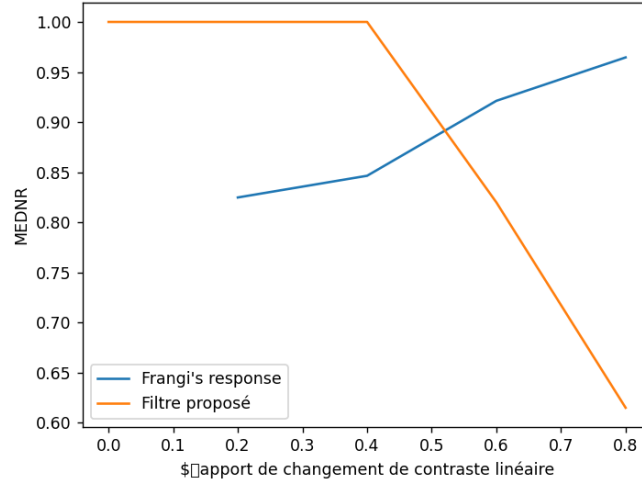


Figure 13: Evolution de MEDNR en fonction du rapport de contraste linéaire

brusque. En résumé, il me semble que les deux réponses ne sont pas uniformes, mais une est plus progressive.

Application 4 : Test de robustesse face au bruit

Notons que contrairement au choix de paramètres, AUPRC peut être pertinent dans ce contexte vu qu'il est influencé par les faux positifs.

Implémentation en 3D

Le filtre de Frangi a été implémenté pour une image 3D en utilisant la formule complète faisant intervenir les 3 valeurs propres. Le format des images utilisés est Nifti, le logiciel utilisé est 3D Slicer. Un résultat a été obtenu pour des vaisseaux pulmonaires. Le résultat paraît globalement satisfaisant bien qu'il paraît aussi confirmer les limitations du filtre de Frangi quant aux bifurcations.

Commentaire : l'analyse est bien plus facile à faire en 2D car on peut voir la donnée brute (impossible en 3D) et que le bruit ne vient pas bloquer notre vision. Une remarque importante est que le temps d'attente du résultat était considérable (entre 10 min et une heure),

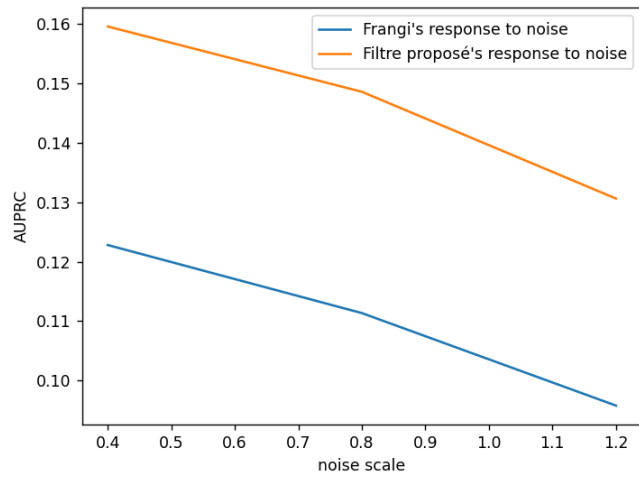


Figure 14: Evolution de AUPRC des réponses en fonction de la variance du bruit blanc ajouté



Figure 15: Résultat de l'implémentation du filtre de frangi en 3D

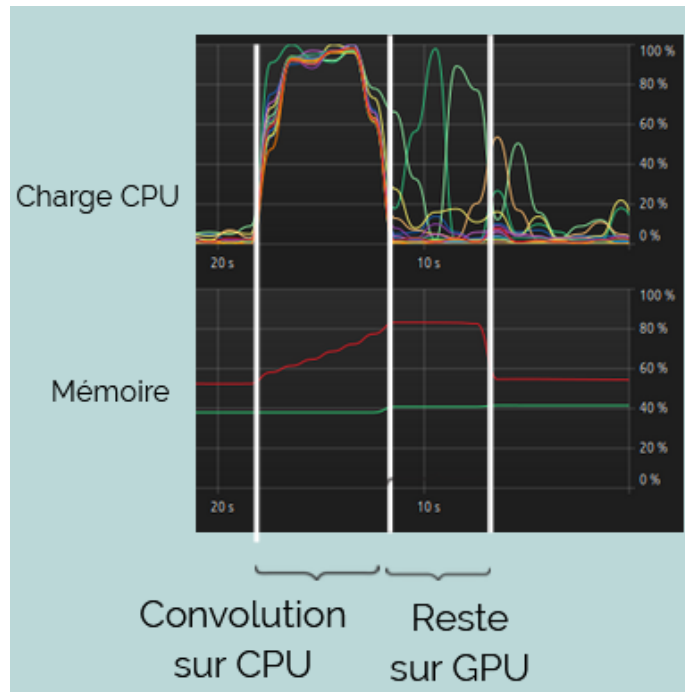


Figure 16: Diminution de la charge ssur CPU après utilisation de la GPU

Implémentation sur GPU

Ce temps de calcul fait le lien avec la dernière partie qui est la programmation GPU. Comme chaque opération dépend surtout de son voisinage, ce problème est massivement parallélisable et donc idéal pour GPU.

Le choix principale à été celui de l'API graphique (ce qui fait le lien entre le code est le GPU). On a choisit OpenGL car déjà fait ouverte et peut être considéré comme open source.

Le code se divise en deux partie. D'abord un code C++ exécuté sur CPU qui instancie le shader, qui lit et écrit dans la mémoire et qui transforme mes donnée nifty en buffer . Ensuite un programme GLSL qui contient le coeur logique du programme qui sera exécuté sur GPU. On obtient les mêmes résultats que en python, mais beacoup plus vite.